



PROCESAMIENTO DE LENGUAJE

NATURAL

Proyecto del Segundo Parcial
Agente conversacional para admisión
y nivelación de la UG

Jorge Santiago Arroyo Chuquín

Odeth Maylin Espinoza Feijoo

Período Académico:

2026 I

1. Introducción al problema abordado

Se conoce que los alumnos de la Universidad de Guayaquil buscan información relacionada con matrícula, nivelación, horarios, admisión y otros procedimientos académicos. Sin embargo, esta información a menudo se encuentra distribuida en diversos medios institucionales, lo que complica su acceso y provoca retrasos, sobre todo cuando la asistencia está sujeta a horarios o al personal administrativo

Para resolver este problema, se creó Agente UG, un agente conversacional que contesta de manera automática las preguntas comunes de la Universidad de Guayaquil. El sistema utiliza métodos de procesamiento del lenguaje natural, incluyendo el preprocesamiento de texto, la representación TF-IDF, la similitud coseno, la detección de intenciones y entidades y un mecanismo de respaldo para preguntas que no se reconocen. Su finalidad es facilitar el acceso a información institucional a través de una interfaz simple y analizar la eficacia de estas técnicas en cuanto a la clasificación y respuesta de preguntas formuladas en lenguaje natural.

2. Diseño de intenciones y entidades

Se creó el agente conversacional con base en las preguntas que se hacen más a menudo acerca de los procedimientos de admisión y nivelación en la Universidad de Guayaquil. Con este propósito, se establecieron propósitos que simbolizan la meta de cada consulta, como los requerimientos para ingresar, las etapas de la admisión, la aceptación de cupos, la matrícula y el nivelamiento. Estas intenciones se guardan en el archivo intents.json, que contiene una etiqueta para cada una de ellas (tag), múltiples patrones de consulta y las respuestas correspondientes, así como una intención secundaria para gestionar preguntas que no pueden ser clasificadas con confianza suficiente.

Se añadieron diversos patrones para cada intención, tomando en cuenta preguntas completas, oraciones breves y diferentes maneras de redactar la misma pregunta, con el objetivo de optimizar la identificación de las consultas. Todos los patrones se procesan por medio de TF-IDF y se comparan con la consulta del usuario a través de la similitud del coseno, lo que posibilita determinar la intención más próxima siempre que supere el umbral de confianza fijado.

El agente también emplea entidades para obtener datos concretos de las consultas, tales como cédulas, nombres de carreras, fechas y términos vinculados con el proceso de admisión. Estas entidades, que se encuentran en el módulo entities.py y fueron implementadas utilizando reglas y expresiones regulares, mejoran la intención detectada y posibilitan brindar respuestas con mayor exactitud. Separar las intenciones de las entidades hace más sencillo conservar el sistema y extenderlo en el futuro al agregar nuevas categorías, patrones y respuestas.

3. Arquitectura del agente conversacional

3.1. Descripción general de la arquitectura

La arquitectura del Agente UG fue creada de manera modular, dividiendo las tareas clave del sistema entre diferentes archivos de Python. La lógica principal del agente se encuentra en el archivo `chatbot.py`, que organiza la carga de las intenciones, la comparación de consultas, la detección de respuestas y el uso del mecanismo de fallback. El módulo `tfidf.py` es responsable de calcular la similitud del coseno y de crear la representación vectorial de los patrones. El módulo `preprocessing.py`, por su parte, se ocupa de normalizar y limpiar el texto. `entities.py` tiene la función de detectar información importante en la consulta. Finalmente, `interfaz.py` ofrece un medio gráfico para interactuar con el usuario. La ampliación, el mantenimiento y la revisión del sistema se simplifican con esta organización.

3.2. Flujo de procesamiento de una consulta

Cuando el usuario introduce una pregunta en la interfaz gráfica, se inicia el flujo. La consulta se remite al módulo principal, que primero utiliza el preprocesamiento lingüístico para conseguir una versión normalizada del mensaje. Más tarde, se convierte el texto procesado en un vector TF-IDF y se coteja con los vectores de patrones guardados en `intents.json` por medio de la similitud del coseno. El sistema escoge el patrón que tiene el valor de similitud más alto, determina la intención vinculada y extrae las entidades que aparecen en la consulta. Si el resultado satisface los requisitos de confianza establecidos, se devuelve una de las respuestas adecuadas; si no es así, se activa el fallback para señalar que la consulta no pudo ser identificada con la seguridad suficiente.

3.3. Módulo de interfaz de usuario

El módulo de interfaz de usuario, que se implementa en el archivo `interfaz.py`, proporciona al alumno la posibilidad de interactuar con el agente a través de una ventana gráfica sencilla. Esta ventana incluye un encabezado institucional, un área para conversar, un campo donde se pueden ingresar preguntas y un botón para enviarlas.

3.4. Módulo de preprocesamiento del lenguaje

El módulo `preprocessing.py`, por su parte, acondiciona los patrones de entrenamiento y las consultas del usuario antes de que se lleve a cabo la comparación. Este procedimiento comprende la normalización de las tildes, la tokenización, el desecho de caracteres e signos de puntuación que no son necesarios, el recorte de palabras vacías y la conversión del texto a minúsculas. También se incluye el acortamiento de términos a través de lematización o stemming. Estos procedimientos reducen las diferencias superficiales entre las oraciones y posibilitan que el modelo compare, sobre todo, las palabras con mayor valor informativo.

3.5. Módulo de representación TF-IDF

El módulo `tfidf.py` se encarga de convertir el texto ya preprocesado en vectores numéricos que el sistema pueda comparar matemáticamente. Primero construye el vocabulario a partir de todas las utterances (frases de ejemplo) del archivo `intents.json`, luego calcula la frecuencia de término (TF) de cada palabra dentro de cada frase, y la frecuencia inversa de documento (IDF), que le da menor peso a las palabras que aparecen en casi todas las intenciones y mayor peso a las palabras distintivas de cada una. El resultado es un vector TF-IDF por cada utterance de entrenamiento, y una función para vectorizar, con el mismo esquema, cualquier consulta nueva que escriba el usuario.

3.6. Cálculo de similitud del coseno

En `chatbot.py`, la función `similitud_coseno()` mide qué tan parecidos son dos vectores TF-IDF calculando el coseno del ángulo entre ellos: $\text{coseno}(A,B) = (A \cdot B) / (\|A\| \times \|B\|)$. El resultado va de 0 (nada parecido) a 1 (idéntico). Se usa coseno en lugar de distancia euclidiana porque mide dirección, no magnitud: dos frases que hablen del mismo tema pueden dar vectores de distinto "tamaño" según el número de palabras, y el coseno sigue reconociéndolas como similares sin que la longitud de la frase afecte el resultado.

3.7. Reconocimiento de intenciones

La función `detectar_intencion()` vectoriza la consulta del usuario y la compara, mediante similitud coseno, contra **todas** las utterances de entrenamiento almacenadas en `intents.json`. Se queda con la intención (tag) de la utterance con mayor similitud, y también devuelve un ranking de las 3 mejores coincidencias, útil para depuración y para el análisis de fallos en la evaluación.

3.8. Extracción de entidades

El módulo `entities.py` complementa la detección de intención extrayendo información específica del dominio dentro de la consulta, usando un enfoque basado en reglas (diccionarios de palabras clave y expresiones regulares), no en machine learning. Reconoce cinco tipos de entidades: ETAPA (etapa del proceso de admisión/nivelación), DOCUMENTO (documento solicitado), PLATAFORMA (sistema institucional mencionado), PERIODO_O_FECHA (año, período o fecha) y PORCENTAJE_O_NOTA (calificaciones o porcentajes). Este enfoque se eligió por ser simple, transparente y suficiente para un dominio acotado y con vocabulario predecible.

3.9. Mecanismo de respuesta y fallback

Sobre el resultado de la similitud coseno se aplica un **umbral de confianza doble**: la similitud máxima debe ser ≥ 0.3 , y deben compartirse al menos 2 tokens entre la consulta y la utterance más parecida (se reduce a 1 token para intenciones conversacionales cortas como saludo/despida). Si cualquiera de las dos condiciones falla, el sistema no arriesga una

respuesta: activa el **fallback**, indicando que no pudo identificar la consulta con la seguridad suficiente. Esto evita falsos positivos, como que una sola palabra casual en común (ej. "Guayaquil") dispare una intención incorrecta.

3.10. Archivos y módulos principales del sistema

EL sistema se encuentra organizado en varios archivos. La lógica básica para manejar los mensajes, determinar la intención más probable, aplicar el umbral de confianza y activar el fallback cuando es necesario se encuentra en el archivo chatbot.py. El módulo tfidf.py, por su parte, se encarga de calcular la similitud del coseno, crear los vectores TF-IDF, construir el vocabulario y cargar las intenciones. En cambio, preprocessing.py se ocupa de normalizar, tokenizar, lematizar y eliminar palabras vacías del texto. Por otro lado, entities.py detecta las entidades importantes en las consultas y interfaz.py se encarga de la interacción con el usuario, la carga del modelo y la gestión de errores a través de estructuras try-except. El archivo intents.json, por otra parte, guarda las respuestas, patrones y etiquetas del agente. El archivo consultas_prueba.csv compila las preguntas que se emplean para analizar su rendimiento.

4. Evaluación del Agente

La evaluación del agente conversacional se llevó a cabo a través de un conjunto de 30 consultas de prueba, las cuales contenían preguntas bien redactadas, errores ortográficos, expresiones breves, variaciones en el lenguaje y mensajes fuera del ámbito institucional. El sistema clasificó de manera correcta 25 de las 30 consultas analizadas, lo que representa un porcentaje de precisión del 83,33 % o una exactitud de 0,8333. Asimismo, alcanzó un F1-macro de 0,8593, lo que muestra un rendimiento global positivo al tener en cuenta de manera equilibrada la precisión y el recall de cada intención. Se observó que varias categorías, incluyendo asignacion_cupos, aceptacion_cupo, etapas_admision, inscripcion_postulacion, cronograma_evaluaciones y registro_nacional, presentaron valores de precisión, recall y F1 de 1. Esto demuestra que los patrones establecidos lograron identificar con exactitud las consultas relacionadas con estos procedimientos.

5. Resultados y análisis

Los resultados por clase muestran que el desempeño no fue uniforme en todas las intenciones. La categoría despedida obtuvo valores de precisión, recall y F1 iguales a 0, debido a que las dos consultas esperadas fueron enviadas al mecanismo de fallback. Por su parte, las intenciones saludo y evaluacion_admision alcanzaron un recall de 0,5, ya que solo una de las dos consultas de cada categoría fue clasificada correctamente. La clase fallback obtuvo una precisión de 0,5, un recall de 0,8 y un F1 de 0,6154, lo que demuestra que el agente identificó la mayoría de las preguntas fuera de su dominio, aunque también activó esta respuesta ante consultas válidas. En contraste, la intención canales_oficiales alcanzó un recall de 1, pero su precisión disminuyó a 0,6667 debido a que una consulta relacionada con la compra de una computadora fue asociada incorrectamente con dicha categoría.

Accuracy: 83.33% (25 de 30 aciertos)

F1-macro: 0.8593 (calculado sobre las 15 intenciones reales, excluyendo "fallback" por no ser una intención real a detectar)

El estudio de los cinco errores hizo posible reconocer tres causas fundamentales: consultas breves, variaciones en el lenguaje y la superposición léxica con preguntas ajenas al dominio. Tres fallos ocurrieron porque frases como "agradezco mucho todo lo explicado", "hasta luego" y "qué contenidos evalúan en el examen de admisión" no fueron identificadas, a pesar de tener valores de semejanza entre 0,5907 y 1. Esto se debió a que el preprocesamiento convirtió cada consulta en un único token informativo y las normas del sistema las remitieron al fallback. La consulta breve "buenas, necesito apoyo con la UG" fue otro caso de error. Su parecido del 0,4597 no bastó para que se la identificara como saludo.

Al compartir términos con los patrones de la intención, la pregunta "dónde puedo comprar una computadora" fue finalmente categorizada como canales_oficiales. Estos hallazgos indican que el agente tiene un funcionamiento apropiado; sin embargo, necesita extender los patrones de saludo, despedida y evaluación de admisión, revisar la eliminación de palabras en el preprocesamiento y modificar las condiciones del fallback para impedir que se declinen consultas válidas con gran similitud.

6. Conclusiones

- El agente logra un desempeño aceptable para un enfoque clásico basado en TF-IDF y similitud coseno, con buena precisión en intenciones bien diferenciadas (saludo, despedida, oferta académica: 100% de F1). Las principales limitaciones detectadas son la sensibilidad a errores tipográficos (por depender de coincidencia exacta de palabras tras el stemming) y la confusión entre intenciones con vocabulario muy similar. Como mejora futura, se podría incorporar corrección ortográfica previa o técnicas de similitud más tolerantes a variaciones (por ejemplo, distancia de edición) para reducir los fallos por typos.
- El agente conversacional probó que la combinación de la representación TF-IDF con la semejanza coseno es una opción efectiva para identificar consultas en un ámbito particular. Estas técnicas, a pesar de no ser algoritmos de clasificación probabilística, posibilitan la asignación de un grado de similitud para cada consulta y la elección de la intención más congruente. El uso de reglas de fallback y de un umbral de confianza contribuyó a regular las respuestas del sistema, pero los resultados mostraron que es necesario modificar estas condiciones para impedir que se desestimen consultas válidas con escasos tokens informativos.
- Con el fin de organizar las preguntas frecuentes relacionadas con la oferta académica, la aceptación de cupos, la asignación, la nivelación y la admisión, se diseñaron las intenciones. Esto fue fundamental para que el agente funcionara correctamente. Las intenciones que tuvieron términos representativos y patrones diversos lograron valores

óptimos de recall, precisión y F1. Sin embargo, las categorías de saludo, despedida y evaluación de admisión mostraron problemas ante expresiones distintas a las registradas, lo cual corrobora que la calidad, variedad y número de patrones tienen un impacto directo en la habilidad del agente para reconocer.

- La evaluación que se llevó a cabo con 30 consultas generó 25 aciertos y 5 errores, lo que resultó en una precisión del 83,33 % y un F1-macro de 0,8593. Estos hallazgos señalan que el agente tiene un rendimiento positivo para una versión inicial y es capaz de responder adecuadamente a la mayoría de las preguntas institucionales evaluadas. No obstante, los fallos asociados a las variaciones del lenguaje, las consultas breves y la superposición de términos demuestran que para aumentar la fiabilidad y exactitud del sistema se requieren mejoras en el preprocesamiento, una ampliación de la base de patrones y un mejoramiento en la detección de preguntas ajenas al dominio.

CUESTIONARIO DE INFORME:

a) ¿Por qué para representar las frases de usuario conviene usar TF-IDF en lugar de una bolsa de palabras con conteos simples? ¿Que aporta el factor IDF al diferenciar unas intenciones de otras?

Una bolsa de palabras simple solo cuenta cuántas veces aparece cada palabra, tratando todas las palabras como igual de importantes. Esto es un problema porque palabras muy frecuentes en casi todas las intenciones (como "admisión", "universidad") tendrían el mismo peso que palabras realmente distintivas (como "nivelación", "cupó", "matrícula"). El factor **IDF** corrige esto: le baja el peso a las palabras que aparecen en muchas utterances distintas (poco informativas para diferenciar) y le sube el peso a las palabras que aparecen en pocas utterances (más informativas y específicas de una intención concreta). Así, TF-IDF permite que la comparación por similitud se apoye en las palabras que realmente distinguen una intención de otra, no en las que se repiten en todo el dominio.

b) Explique por qué para comparar la consulta con las utterances utilizó la similitud coseno y no la distancia euclidiana. ¿Qué problema resuelve la normalización de longitud cuando las frases tienen distinto número de palabras?

La similitud coseno mide el **ángulo** entre dos vectores, no su magnitud (tamaño). Por eso es independiente de la longitud del texto: una consulta corta ("cupó asignado") y una larga que hable del mismo tema ("por qué no me asignaron un cupo en la carrera que escogí") tendrán vectores TF-IDF de distinto tamaño solo por tener más o menos palabras, pero si comparten los términos relevantes y apuntan en una dirección similar, el coseno las reconoce como parecidas.

La distancia euclidiana, en cambio, sí se ve afectada por esa diferencia de longitud: penalizaría a la frase más larga aunque hable exactamente del mismo tema, porque mide la distancia "recta" entre los puntos, no su orientación. Por eso el coseno resuelve el problema de la normalización de longitud: permite comparar consultas de distinto tamaño contra las utterances de entrenamiento sin que la cantidad de palabras distorsione el resultado.

c) ¿Por qué aplicó preprocesamiento (lematización y eliminación de stopwords) antes de vectorizar? De un ejemplo, propio de su proyecto, de cómo ese paso modifica el vocabulario y mejora la coincidencia en español.

Se utilizó el preprocesamiento para disminuir las variaciones lingüísticas que no alteran el significado principal de las consultas antes de crear los vectores TF-IDF. La lematización convierte palabras flexionadas a una forma estándar, en cambio la eliminación de stopwords descarta términos que aparecen con frecuencia, como pronombres, preposiciones y artículos, que suelen aportar información escasa para determinar una intención. Así, el vocabulario que

emplea TF-IDF está constituido sobre todo por términos pertinentes para el ámbito de admisión y nivelación de la Universidad de Guayaquil.

Un ejemplo de esto es que la pregunta del proyecto "¿Cuáles son los requisitos para ingresar a la UG?", tras la lematización, eliminación de palabras vacías y normalización, podría transformarse en una secuencia parecida a "requisito ingresar ug". Las palabras "cuáles", "son", "los", "para" y "a" se descartan por tener escaso valor discriminativo; en cambio, la palabra "requisitos" se reduce a su forma base, que es "requisito". Esto posibilita que la consulta coincida mejor con patrones como "requisitos de ingreso a la universidad", dado que los textos comparten palabras fundamentales como "requisito" e "ingresar", aunque se hayan escrito de manera distinta.

d) Justifique el uso de un umbral de confianza y de la respuesta de fallback: ¿Cómo eligió el valor del umbral y qué ocurriría con las consultas fuera de alcance si no existiera ese umbral?

Con el fin de prevenir que el agente diera una respuesta incorrecta si la consulta del usuario coincidía muy poco con los patrones guardados, se usó el umbral de confianza. Se definió, a través de pruebas con preguntas externas y consultas del dominio, un límite inicial de 0.30 en el proyecto. Se buscó un valor que hiciera posible aceptar distintas formas de hacer una consulta, sin necesidad de que el sistema tuviera que asociar cualquier texto con una intención institucional. El sistema también tuvo en cuenta, aparte de la similitud, normas que tenían que ver con el número de tokens informativos de la consulta.

Cuando ninguna coincidencia satisface los requisitos mínimos establecidos, se activa la respuesta de fallback. Su labor es señalar al usuario que la pregunta no fue entendida o que excede el alcance del agente. Sin este umbral, el sistema siempre escogería el patrón matemáticamente más próximo, aun cuando la correlación no fuera correcta o fuese débil. Por ejemplo, una pregunta como "¿En qué lugar puedo adquirir un ordenador?" podría vincularse con canales oficiales únicamente por tener palabras en común que se relacionan con sitios o canales de información. Del mismo modo, cuestiones acerca del clima, el cine o los deportes podrían ser respondidas con admisión, a pesar de que estén totalmente fuera del dominio.

Los resultados de la evaluación también indicaron que es necesario modificar con cuidado las reglas de fallback y el umbral. Algunas preguntas válidas, como "ya resolví mi duda, hasta luego", fueron enviadas al fallback a pesar de tener una alta similitud, porque después del preprocesamiento quedaron pocos tokens informativos. Por lo tanto, el valor del umbral debe evaluarse no de forma independiente, sino en combinación con la calidad de los patrones, la longitud de la consulta y las reglas complementarias de aceptación.

e) ¿Por qué almacenó las intenciones, utilidades y respuestas en un archivo JSON/CSV separado del código? ¿Qué ventaja de diseño aporta separar los datos de la lógica del agente?

Para conservar separada la lógica de funcionamiento del agente y los datos de entrenamiento, las respuestas, las intenciones y los patrones de consulta se guardaron en un archivo JSON distinto al código. En `intents.json`, cada categoría está estructurada mediante un tag y dos listas: una de `patterns` y otra de `responses`. Los archivos de Python, además, son responsables de llevar a cabo el preprocesamiento, la construcción de los vectores TF-IDF, el cálculo de la similitud, la extracción de entidades y la elección de la respuesta pertinente.

Esta división proporciona escalabilidad, mantenibilidad y modularidad. Para ilustrar, para agregar una nueva intención acerca de las fechas de matrícula o incluir otros modos de consultar sobre la aceptación de cupos, basta con cambiar el archivo JSON sin tener que modificar el algoritmo principal. Esto disminuye la posibilidad de que se introduzcan errores en el código y permite que un responsable del contenido institucional tenga la capacidad de actualizar preguntas y respuestas sin requerir habilidades programáticas avanzadas.

Además, posibilita que se use la misma lógica del agente con diversos conjuntos de conocimiento. El sistema podría ser capaz de adaptarse a otro departamento universitario, reemplazando o ampliando el archivo de intenciones y manteniendo los módulos de preprocesamiento, TF-IDF y similitud coseno. Por lo tanto, al separar los datos de la lógica se logra una estructura más ordenada y se favorece el crecimiento del proyecto en el futuro.

f) ¿Cómo manejó con try-except una entrada problemática (consulta vacía, texto no reconocido o con errores tipográficos) sin que el agente se detenga?

Para impedir que una entrada problemática detenga la ejecución del programa, el agente se vale de validaciones y estructuras try-except. La condición `if not consulta` verifica que no hay contenido válido si el usuario ingresa una consulta vacía o solo con espacios, y la instrucción `input().strip()` suprime los espacios. En tal situación, el sistema presenta el mensaje "Por favor escribe una consulta" y ejecuta `continue`, volviendo al comienzo del ciclo para pedir otra entrada sin cerrar el agente.

Las consultas que no son reconocidas o contienen errores tipográficos se mandan a la función `procesar_mensaje()`, en la cual se comparan con los patrones usando TF-IDF y similitud coseno. Si la consulta no llega al nivel de confianza necesario, el sistema utiliza la respuesta de contingencia. Los fallos ortográficos no interrumpen el programa de forma directa, pero sí pueden disminuir la semejanza con los patrones guardados. El bloque `except Exception as error` recoge y presenta el mensaje "No fue posible procesar la consulta" si se produce una excepción no anticipada mientras se procesa, extrae entidades o genera la respuesta. "Intenta reformularla", de modo que el ciclo `while` siga y el usuario tenga la posibilidad de hacer otra pregunta.

Asimismo, el código regula otras circunstancias que podrían detener la acción del agente. `FileNotFoundError` identifica si falta un archivo necesario durante la carga inicial, mientras que otro `except Exception` se encarga de manejar los errores en la construcción del modelo TF-IDF. Cuando el usuario detiene manualmente la ejecución, también se capturan

KeyboardInterrupt y EOFError. Por lo tanto, el agente utiliza validaciones, fallback y gestión de excepciones para reaccionar de forma controlada ante entradas vacías, textos desconocidos, fallos tipográficos o errores imprevistos, sin que la conversación termine de manera brusca.